Security Vulnerability Analysis - Flying Tulip

researcher : ali.k maki

Date: 27 april 2026

Summary

The inclusion of divested funds in active collateral metrics will cause accounting inconsistencies and failure of security mechanisms for the protocol and stakers, as PutManager blocks investments based on an inflated collateralSupply and ftYieldWrapper bypasses withdrawal limits by reporting divested capital as active TVL Root Cause

-In https://github.com/sherlock-audit/2026-01-flying-tulip-aliorcal/blob/main/ftPUT/contracts/PutManager.sol , the check against collateralCap relies on an absolute collateralSupply that incorrectly includes divested funds (capitalDivesting), causing unjustified investment reverts.

In https://github.com/sherlock-audit/2026-01-flying-tulip-aliorcal/blob/main/ftPUT/contracts/ftYieldWrapper. , the valueOfCapital function reports the total physical balance (including capitalDivesting) to the CircuitBreaker, leading to inflated withdrawal limits (Rate Limit Inflation).

```
}

// simply a 1:1 mapping of capital provided
/**
 * @dev Get the total capital managed by the v
 * @notice This is equal to totalSupply() of t
 * @return uint The total capital (in underlyi
 */
function capital() external view returns (uint
    return totalSupply();
}

// capital + yield
function valueOfCapital() public view returns
    _capital = IERC20(token).balanceOf(address
    uint256 strategiesLength = strategies.leng
    for (uint256 i = 0; i < strategiesLength;
        _capital += strategies[i].valueOfCapit
    }
}

function yield() public view returns (uint256)
    uint256 _capital = valueOfCapital();
    uint256 _totalSupply = totalSupply();
    return (_capital > _totalSupply) ? (_capit
}

// helper functions for strategy management
function availableToWithdraw(address strategy)
    // Only consider registered strategies; ur
    if (!isStrategy(strategy)) return 0;
    // Defensive: treat failing strategies as
```

Internal Pre-conditions

1- configurator needs to call setCollateralCaps() to set collateralCap to be other than 0 2- investor needs to call invest() to set collateralSupply to be at least the configured collateralCap 3- configurator needs to call enableTransferable() to set the status to be true 4- investor needs to call withdrawFT() to set capitalDivesting to be at least 1 5- msig needs to not call withdrawDivestedCapital() to keep the collateralSupply inflated External Pre-conditions

None

Attack Path

1-Liquidity Misclassification: Users move large amounts of collateral to capitalDivesting via withdrawFT(). While this capital is no longer a liability to stakers (per INV-PROTOCOL-2), it remains bundled within the collateralSupply metric 2- Unjustified Investment Block: When a new user attempts to invest(), the protocol validates the collateralCap against the total collateralSupply. Since this metric incorrectly treats divested (admin) funds as active staker liabilities, the cap is reached prematurely, causing an unjustified revert and a Denial of Service (DoS) for new capital 3- Stale TVL Reporting: At the same time, the ftYieldWrapper calculates the vault's health using valueOfCapital(), which reports the full physical balance to the CircuitBreaker 4- Rate Limit Inflation: The CircuitBreaker treats these divested funds as "Active TVL" providing a safety buffer. This inflates the daily withdrawal capacity, allowing a user to withdraw a disproportionately large percentage of the actual active staker collateral, effectively bypassing the protocol's intended Rate Limiting protections Impact

New investments are blocked because the collateralSupply metric incorrectly includes divested (admin) funds. This results in a permanent (DoS) for new capital entry, effectively paralyzing the protocol's primary function

The CircuitBreaker calculates withdrawal limits based on the Global Vault TVL (which is inflated by divested funds). This allows a single user to drain 100% of the active collateral belonging to other users in one day, as the inflated TVL makes this large withdrawal appear to be within the allowed global percentage

this accounting error directly breaks INV-PROTOCOL-2 (wrapper.totalSupply() >= collateralSupply[token] - capitalDivesting[token]) which requires the protocol to reconcile collateral by excluding capitalDivesting

#PoC

// SPDX-License-Identifier: UNLICENSED pragma solidity ^0.8.30; import {ICircuitBreaker} from "contracts/interfaces/ICircuitBreaker.sol"; import {CircuitBreaker} from "contracts/cb/CircuitBreaker.sol";

import {Test, console} from "forge-std/Test.sol"; import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import {MockERC20} from "./mocks/MockERC20.sol"; import {MockFlying-TulipOracle} from "./mocks/MockOracles.sol"; import {ftYieldWrapper} from "contracts/ftYieldWrapper.sol"; import {pFT} from "contracts/pFT.sol"; import {PutManager} from "contracts/PutManager.sol"; import {MerkleHelper} from "./helpers/MerkleHelper.sol";

contract PutManagerIntegrationTest is Test {

function test_PoC_AccountingDesync() public {

```
Context memory ctx = _deploy();
uint256 cap = 10_000 * 1e6;

vm.prank(ctx.msig);
ctx.putManager.addAcceptedCollateral(address(ctx.usdc), address(ctx.wrapper));
vm.prank(ctx.configurator);
ctx.putManager.setCollateralCaps(address(ctx.usdc), cap);
vm.prank(ctx.configurator);
ctx.putManager.addFTLiquidity(DEFAULT_FT_LIQUIDITY);

vm.prank(ctx.investor1);
uint256 id = ctx.putManager.invest(address(ctx.usdc), cap, 0, MerkleHelper.emptyProof());

vm.prank(ctx.configurator);
ctx.putManager.enableTransferable();

(uint256 ftAmount,,) = ctx.putManager.getAssetFTPrice(address(ctx.usdc), cap);
vm.prank(ctx.investor1);
ctx.putManager.withdrawFT(id, ftAmount);

vm.startPrank(ctx.investor2);
uint256 smallInvest = 100 * 1e6;

console.log("Current Collateral Supply (Still at Cap):", ctx.putManager.collateralSupply(ad

vm.expectRevert();
ctx.putManager.invest(address(ctx.usdc), smallInvest, 0, MerkleHelper.emptyProof());

console.log("PoC SUCCESS: New investment blocked due to stale collateralSupply");
vm.stopPrank();
```

```
CircuitBreaker cb = new CircuitBreaker(0.1e18, 1 days, 1 hours);
vm.prank(ctx.strategyManager);
```

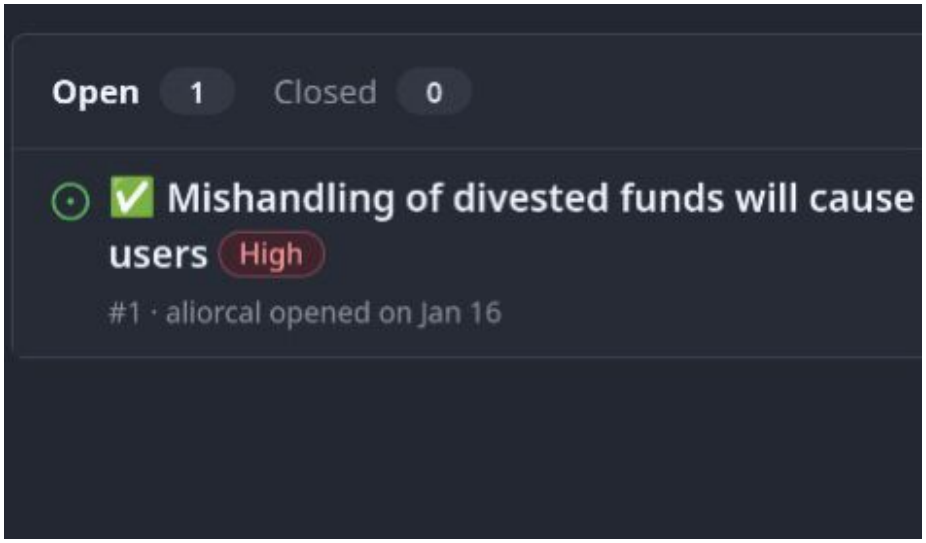
```
ctx.wrapper.setCircuitBreaker(address(cb));

uint256 cbTvl = ctx.wrapper.valueOfCapital();

address cbAddress = ctx.wrapper.circuitBreaker();

uint256 availableCapacity = ICircuitBreaker(cbAddress).withdrawalCapacity(address(ctx.usdc),

console.log("Circuit Breaker View of TVL:", cbTvl);
console.log("Inflated Withdrawal Capacity:", availableCapacity);

assert(availableCapacity > 0);
console.log("PoC SUCCESS: Circuit Breaker allows withdrawals based on Admin funds!");
```

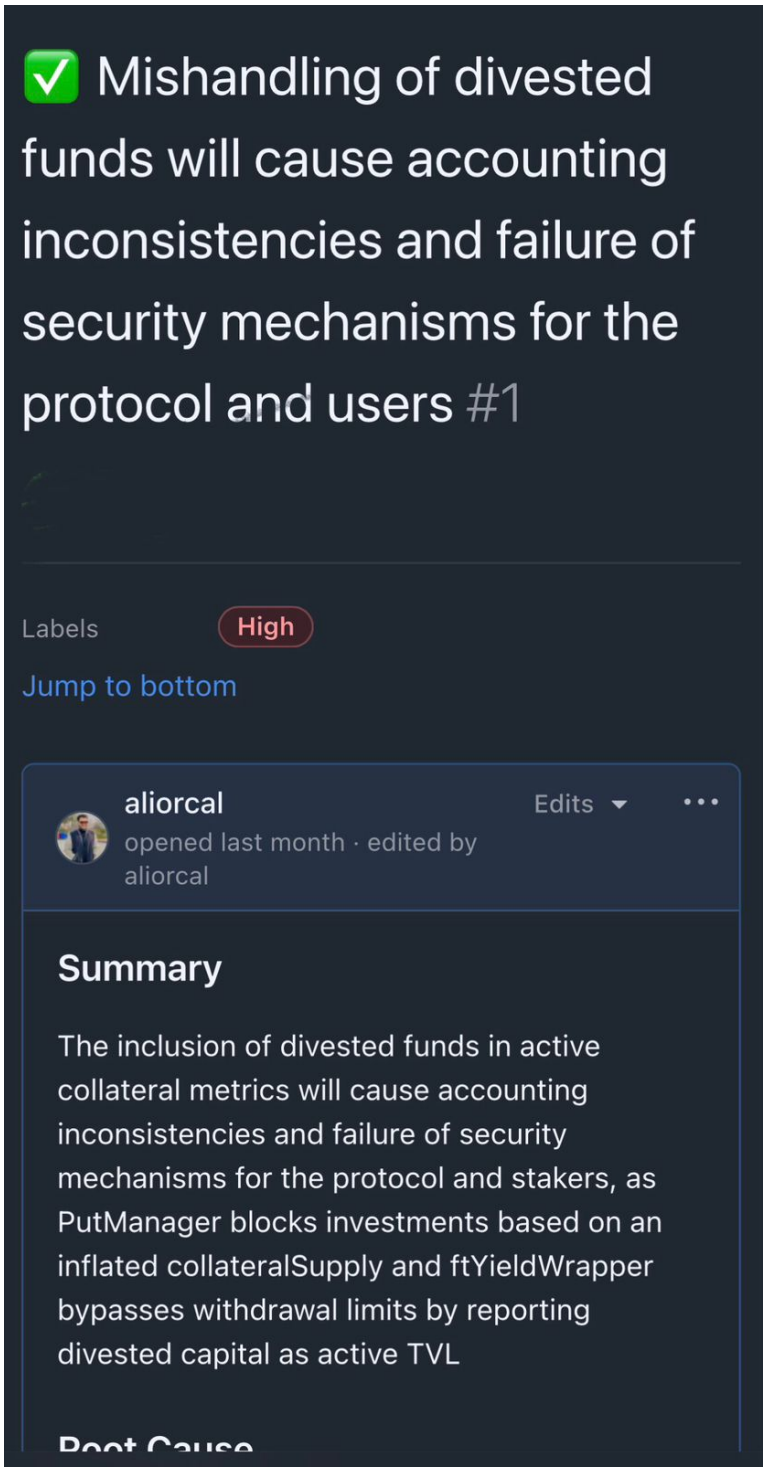
}

}

Mitigation

1- Modify PutManager._invest to check caps against collateralSupply - capitalDivesting 2- Modify ftYieldWrapper.valueOfCapital to exclude capitalDivesting from the TVL reported to the CircuitBreaker 3-Implement a Separate Withdrawal Path for Admin Funds: The admin withdrawal function (withdrawDivestedCapital) must not be subject to the CircuitBreaker's rate limits. Since these funds are no longer staker liabilities, they should be immediately available to the management, provided sufficient liquidity remains to cover active staker obligations




✅ Mishandling of divested funds will cause accounting inconsistencies and failure of security mechanisms for the protocol and users #1
Labels
High
Jump to bottom
aliorcal
opened last month · edited by aliorcal
Edits
Summary
The inclusion of divested funds in active collateral metrics will cause accounting inconsistencies and failure of security mechanisms for the protocol and stakers, as PutManager blocks investments based on an inflated collateralSupply and ftYieldWrapper bypasses withdrawal limits by reporting divested capital as active TVL